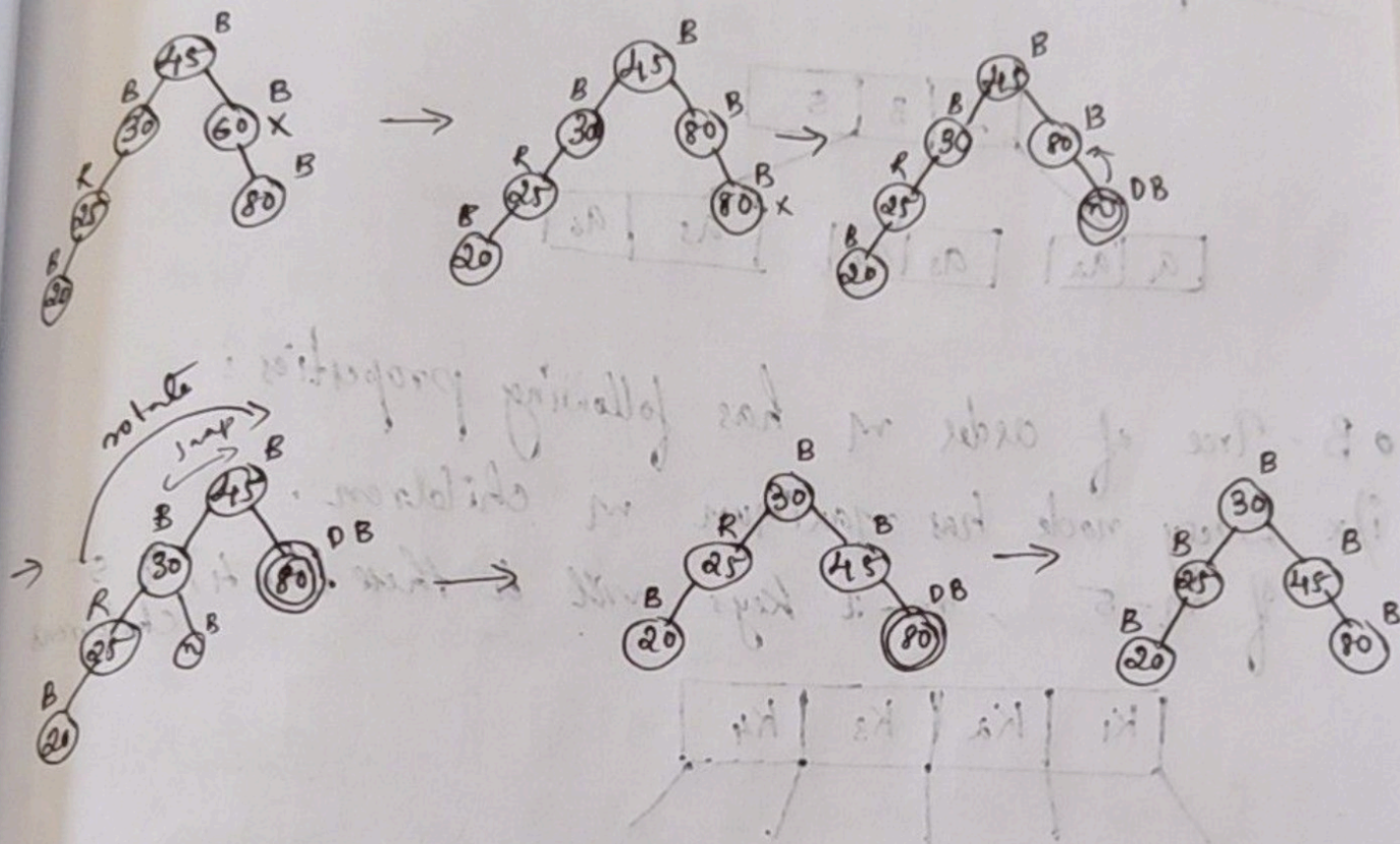


→ Delete 60



B-Tree

→ order

- Balanced M-way tree
- Generalization of BST in which a node can have more than 2 key and more than 2 children.
- Maintains sorted data.

Node is represented by

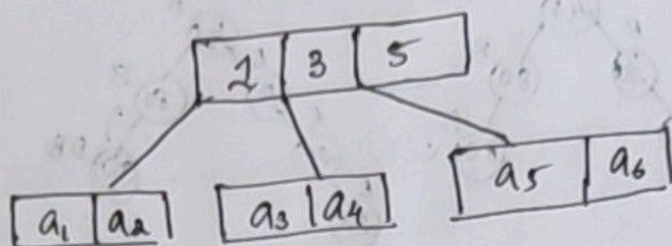
20	21
----	----

 or

20	
----	--

- All leaf node must be at same level.

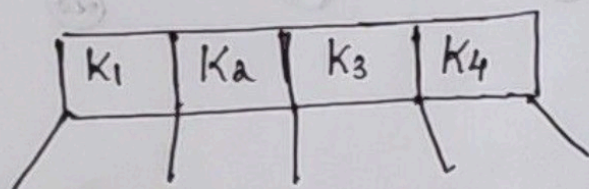
• Example



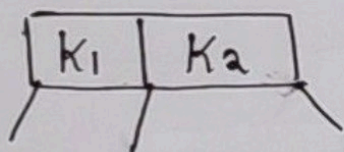
• B-Tree of order m has following properties:

i) * Every node has maximum m children.

If $m=5$, $m-1$ keys will be there (here 4) & 5 children



If $m=3$, no. Key = $m-1=2$, children = 3



ii) * Minimum children $\rightarrow$ leaf = 0

Root = 2

Internal node = $m/2$

Eg: If $m=5$, no. of keys = $m-1=4$, children = 5, [maximum]

Eg: [Minimum] $\rightarrow$ leaf = 0, Root = 2, internal = $\frac{5}{2} = 2.5 \rightarrow 3$ [ceiling value].

ii) * Min keys : Root = 1

All other nodes = $\lceil m/2 \rceil - 1$

Eg: If $m=6$

max no. of keys = $m-1 = 5$

max no. of children = 6

min no. of leaf = 0

min root = 2

min no. of internal node = $m/2 = 6/2 = 3$

min keys = 1

All other nodes = $\lceil m/2 \rceil - 1 = 3 - 1 = 2$

Insertion

Q). Construct a B-Tree of order 4 [$m=4$].

Keys $\Rightarrow$ 5, 3, 21, 9, 1, 13, 2, 7, 10, 12, 4, 8

Ans: Here $m=4$

$\Rightarrow$ max no. of keys = $m-1 = 3$

$\Rightarrow$ max no. of children = 4

$\Rightarrow$ min no. of leaf = 0

$\Rightarrow$ min root / children = 2

$\Rightarrow$ All other nodes = $\lceil m/2 \rceil - 1 = \lceil 4/2 \rceil - 1 = 2 - 1 = 1$

$\Rightarrow$ min no. of internal node = $m/2 = 4/2 = 2$

Adding the keys to the B-tree structure. The root node has 4 pointers to 4 leaf nodes. The keys are distributed among the leaf nodes as follows: Leaf 1: 5, 3, 21; Leaf 2: 9, 1, 13; Leaf 3: 2, 7, 10; Leaf 4: 12, 4, 8.

iii) * Min keys : Root = 1
 All other nodes = $\lceil n/2 \rceil - 1$

Eg: If $m=6$

max no. of keys = $m-1 = 5$

max no. of children = 6

min no. of leaf = 0

min root = 2

min no. of internal node = $n/2 = 6/2 = 3$

min keys = 1

All other nodes = $\lceil n/2 \rceil - 1 = 3 - 1 = 2$

Insertion

Q). Construct a B-Tree of order 4 [$m=4$].

Keys $\Rightarrow$ 5, 3, 21, 9, 1, 13, 2, 7, 10, 12, 4, 8

Ans: Here $m=4$

$\Rightarrow$ max no. of keys = $m-1 = 3$

$\Rightarrow$ max no. of children = 4

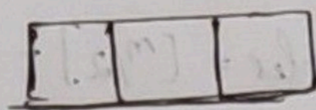
$\Rightarrow$ min no. of leaf = 0

$\Rightarrow$ min root / children = 2

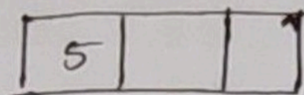
$\Rightarrow$ All other nodes = $\lceil \frac{m}{2} \rceil - 1 = \lceil \frac{4}{2} \rceil - 1 = 2 - 1 = 1$

$\Rightarrow$ min no. of internal node = $n/2 = 12/2 = 6$

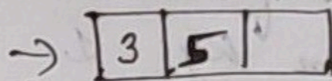
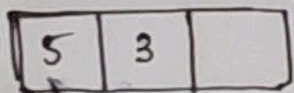
Adding the keys to the B-tree structure. The keys are 5, 3, 21, 9, 1, 13, 2, 7, 10, 12, 4, 8. The B-tree structure is shown below. The root node has 4 pointers to 4 leaf nodes. The leaf nodes contain the keys in sorted order. The root node contains the keys 5, 3, 21, 9. The leaf nodes contain the keys 1, 2, 7, 10, 12, 4, 8, 13, 21, 5, 3.



⇒ Insert 5:



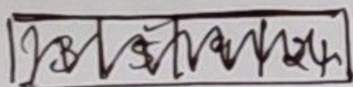
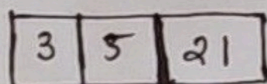
⇒ Insert 3:



make it sorted

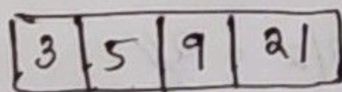
[nodes should be sorted]

⇒ Insert 21:

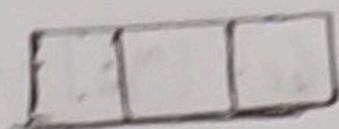


⇒ Insert 9:

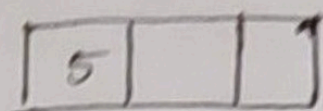
9 can be inserted after 5 & before 21.



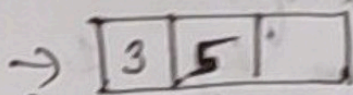
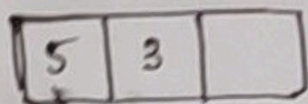
Now the node contain 4 keys → Not possible
 So we need to split the node. Splitting will be done using median elements



⇒ Insert 5:



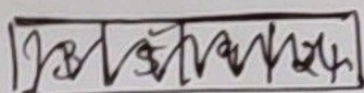
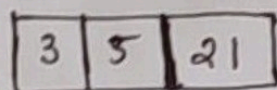
⇒ Insert 3:



make it sorted

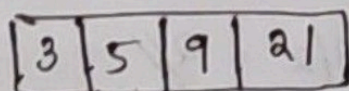
[nodes should be sorted]

⇒ Insert 21:

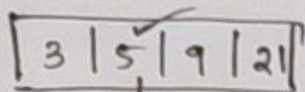


⇒ Insert 9:

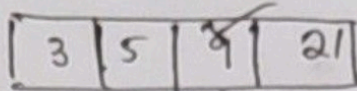
9 can be inserted after 5 & before 21.



Now the node contain 4 keys → Not possible
 So we need to split the node. Splitting will be done using median elements



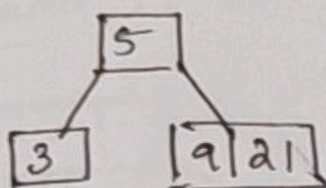
Left biased



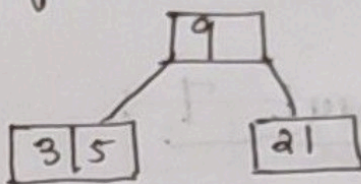
Right biased

Here median elements are 5 & 9, either you can select 5 or 9. Left element (5) → Left biased.

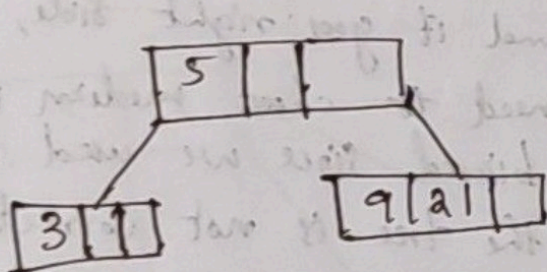
⇒ If we choose 5



⇒ If we choose 9

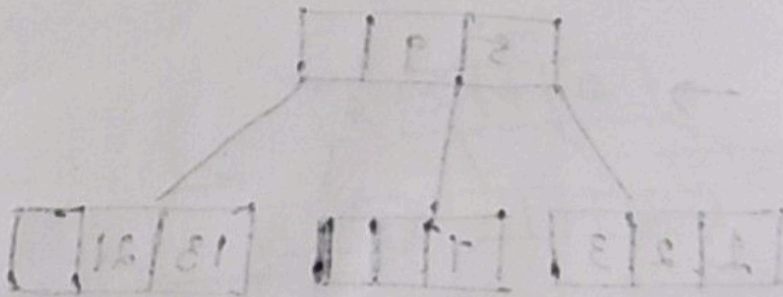
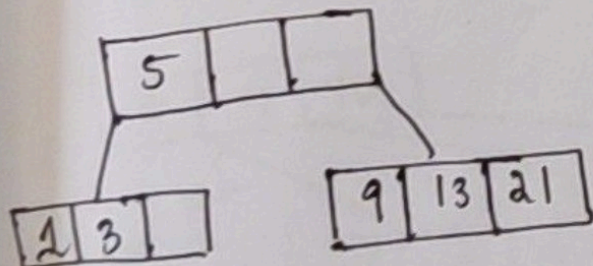


⇒ Insert 1

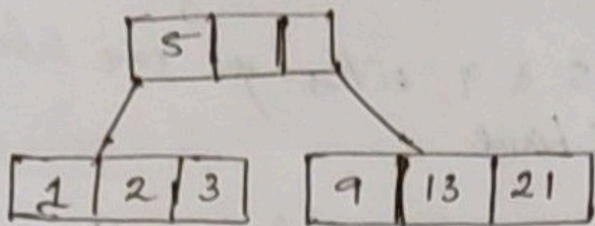


Here $1 < 5$ → So goes to left side.

⇒ Insert 13 :

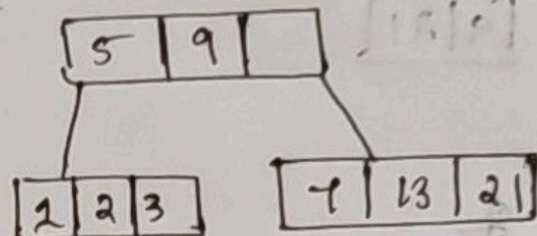
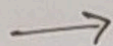
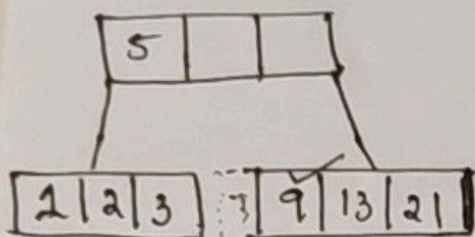


⇒ Insert 2

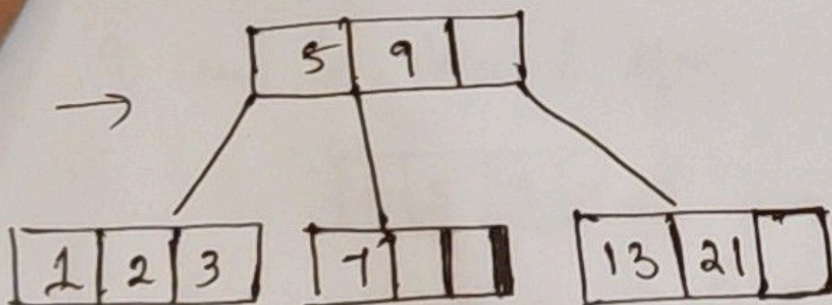


we compared two with root node (5). so it goes to left side.

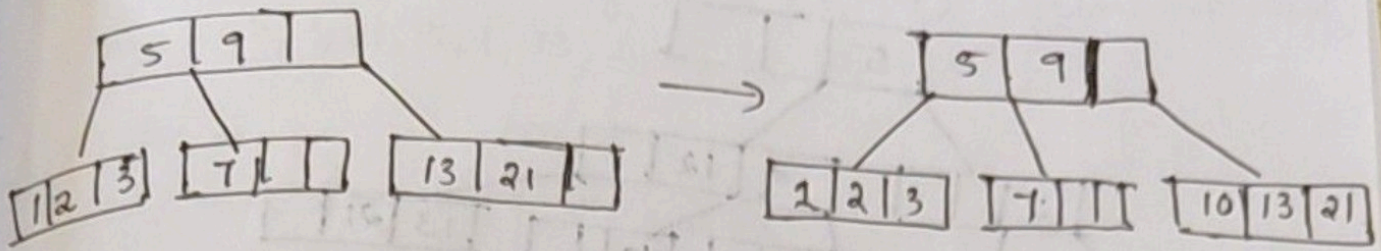
⇒ Insert 7



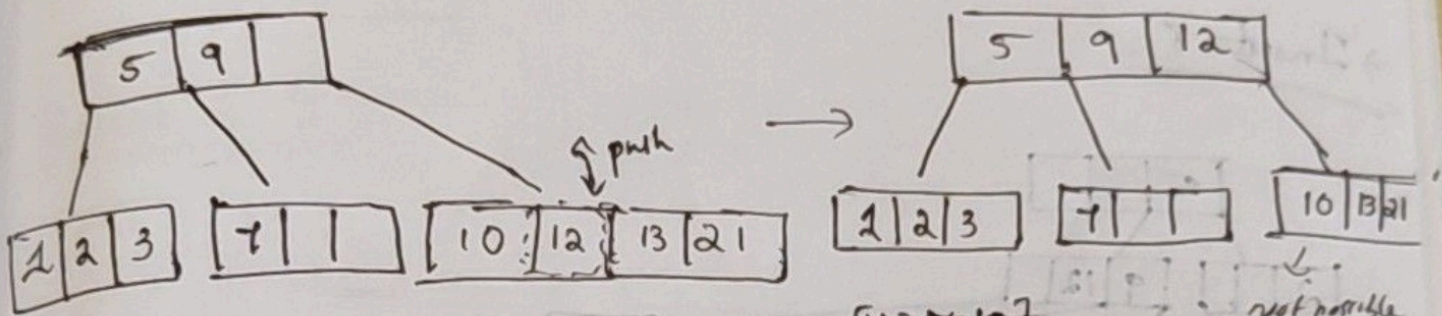
Here we compared 7 with 5 and it goes right side, then there becomes 4 keys, so we need to choose median nodes, here we should choose left biased since we used that earlier. So 9 goes up. Still the tree is not correct, since 7 is lesser than 9. So we create separate key.



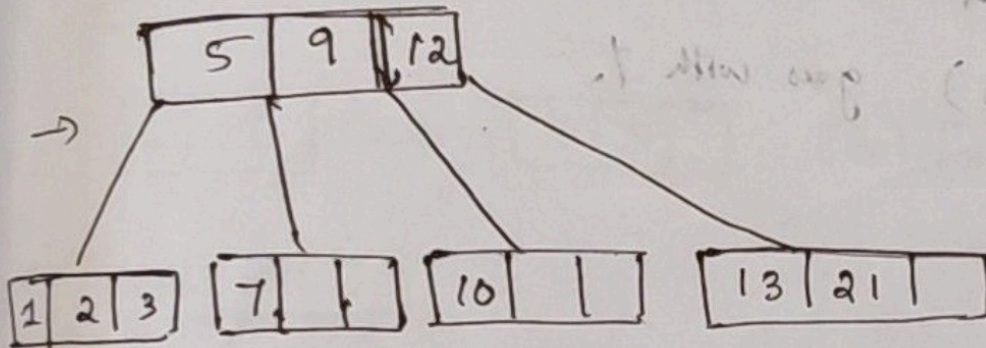
⇒ Insert 10 :



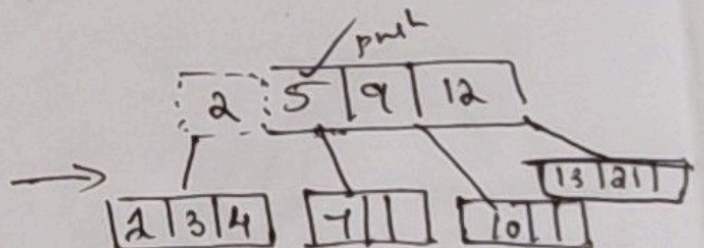
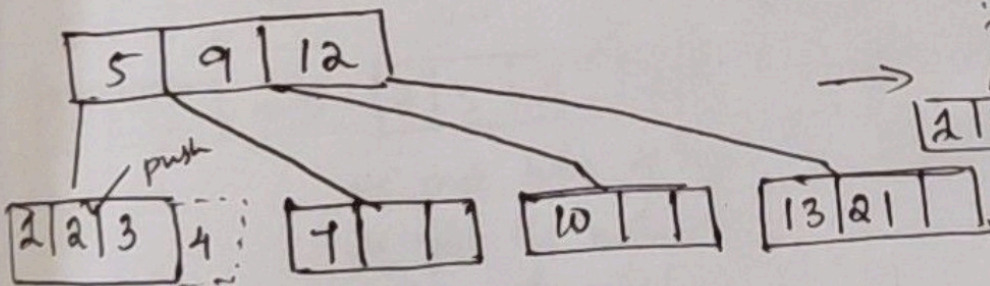
⇒ Insert 12 :



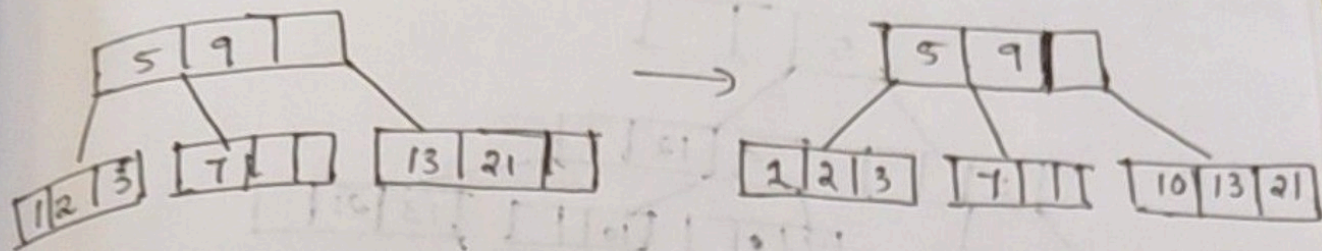
(max 10) $10 > 12$ not possible
So it



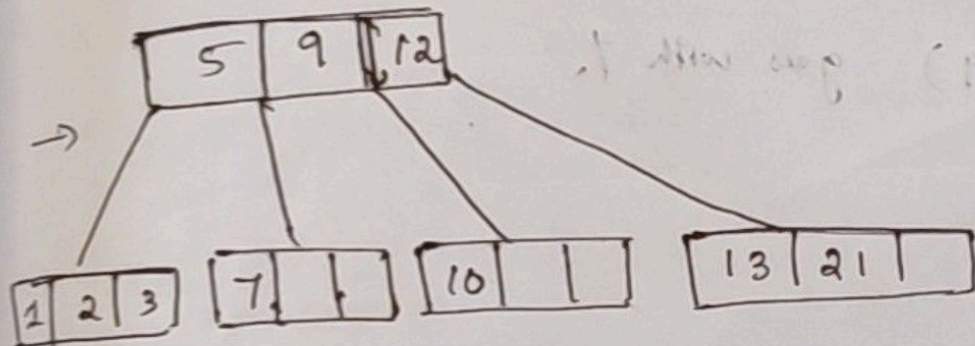
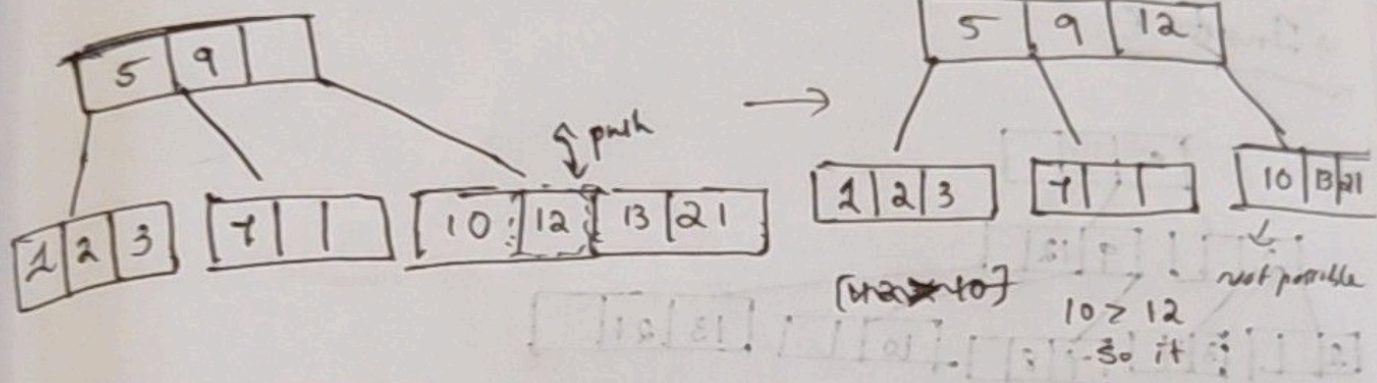
⇒ Insert 4 :



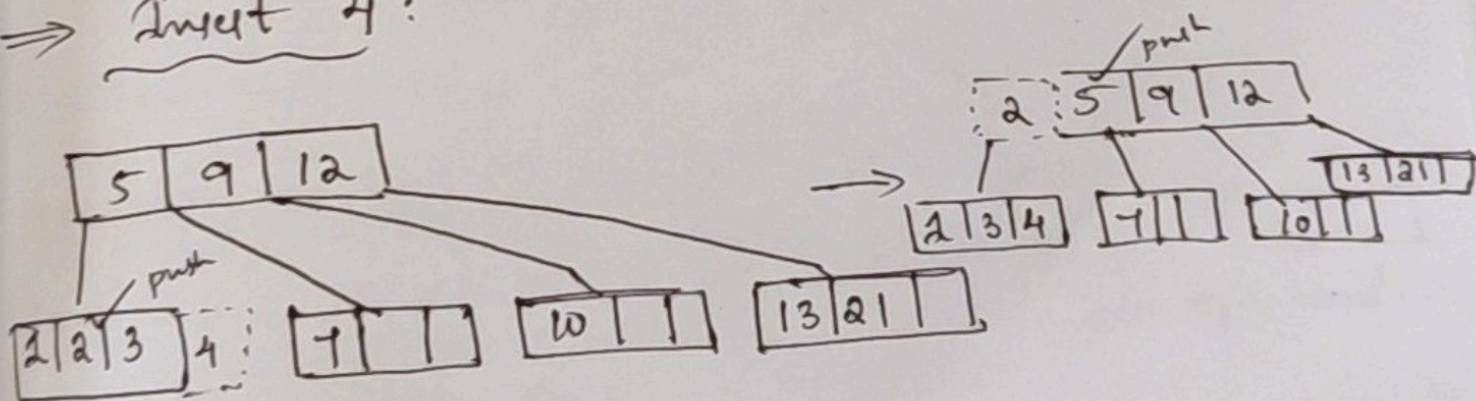
⇒ Insert 10 :

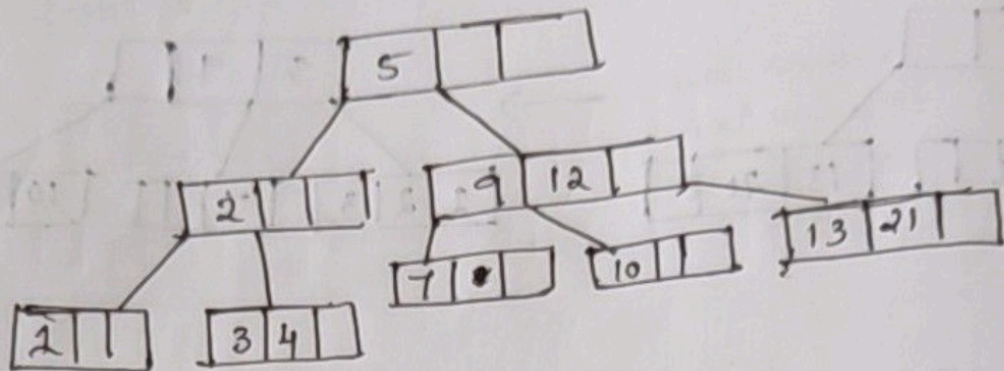


⇒ Insert 12 :

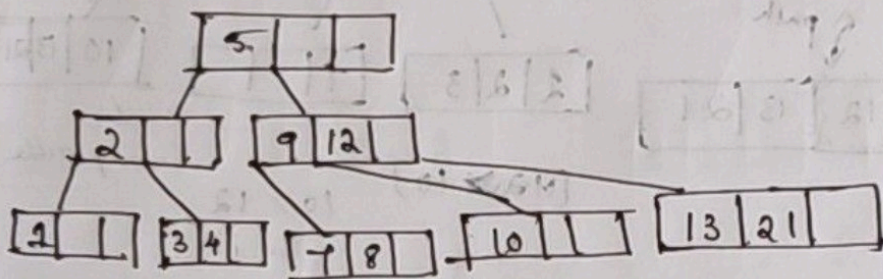


⇒ Insert 4 :





→ Insert: 8:



$(8 > 5 \ \& \ 8 < 9)$ goes with 7.

Q. Construct a B-Tree of order 4 [m=4] [choose right biased]

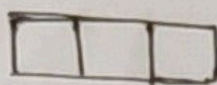
Keys $\Rightarrow 5, 3, 21, 9, 1, 13, 2, 7, 10, 12, 4, 8.$

Ans: Here $m=4$

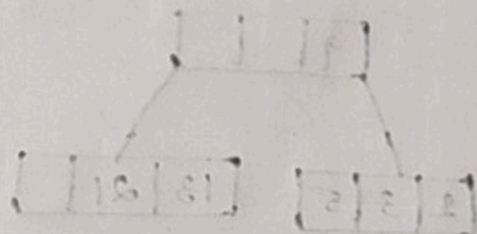
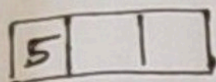
$\Rightarrow$ max no. of keys $= M-1 = 4-1 = 3$

$\Rightarrow$ max no. of children $= 4$

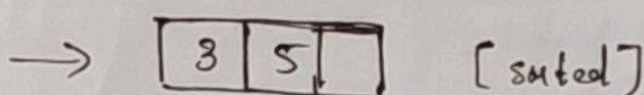
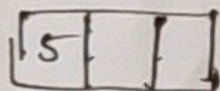
$\Rightarrow$ min no. of children $= 2$



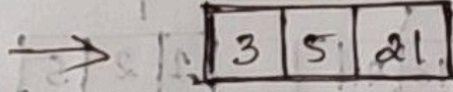
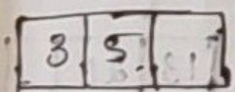
* Insert 5 :



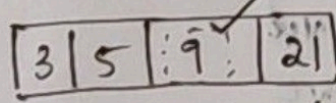
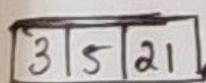
* Insert 3 :



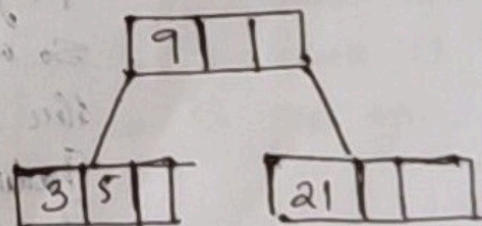
* Insert 21 :



* Insert 9 :

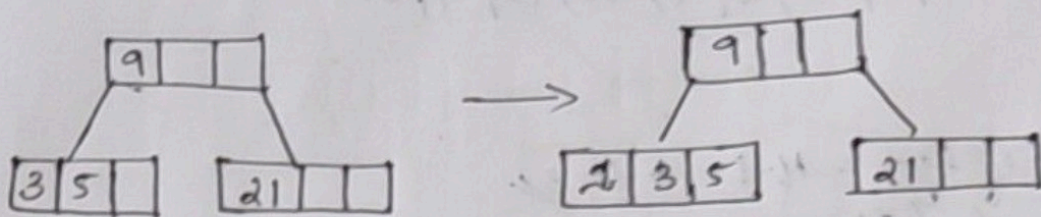


we can't have 4 keys
So split the node using
median elements [here 5 & 9]



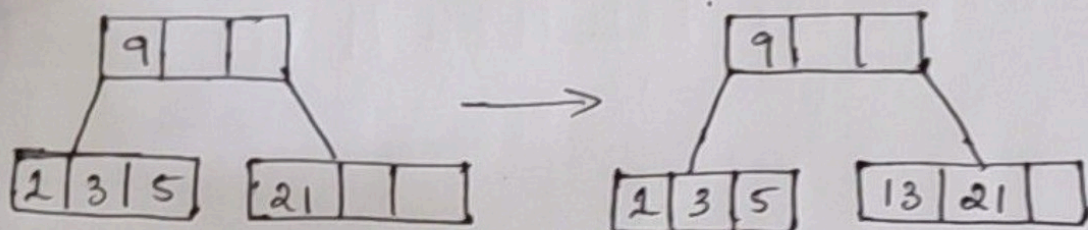
we choose right biased.

* Insert 1:



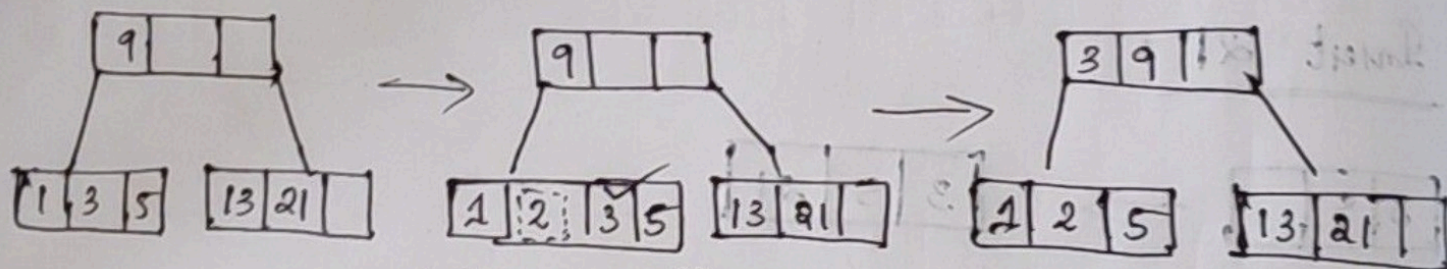
Here $1 < 9$ so goes to left side

* Insert 13:



$13 > 9$ so it goes to right side.

* Insert 2:



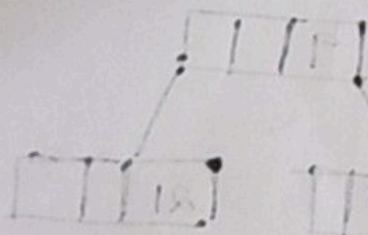
4 keys not possible

So we split nodes.

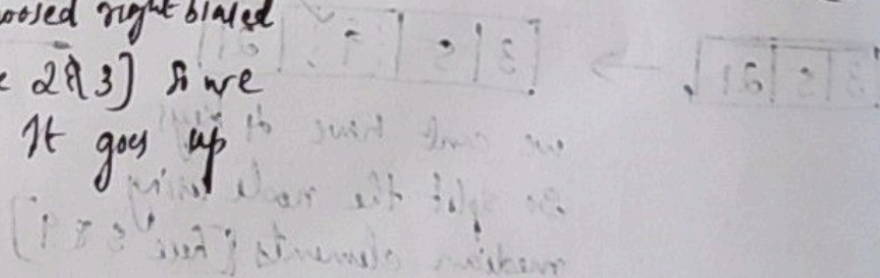
Here we choose right biased

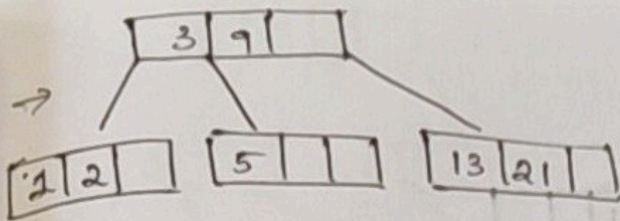
(medians are 2 & 3) so we choose 3. It goes up

Here $5 > 3$, so create new children node.

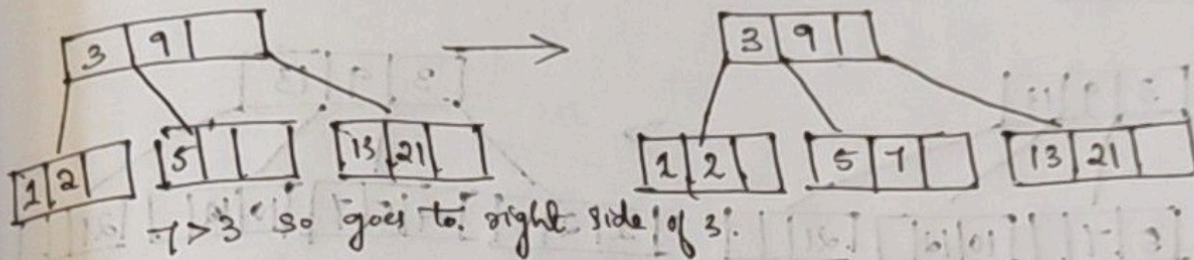


... (faint text)

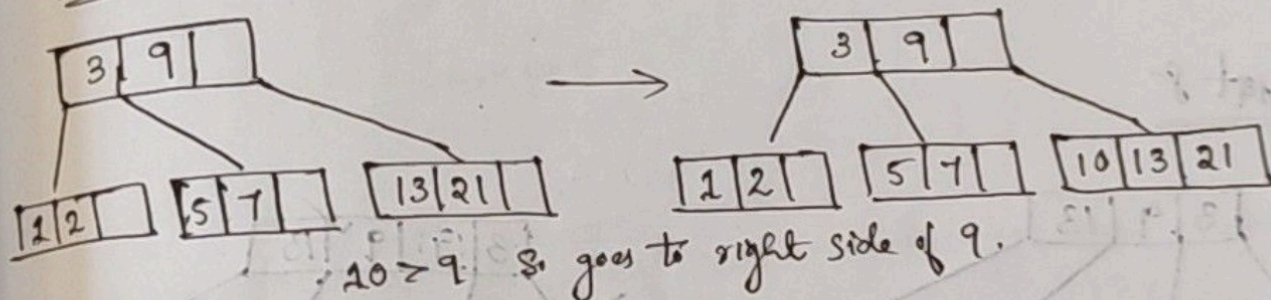




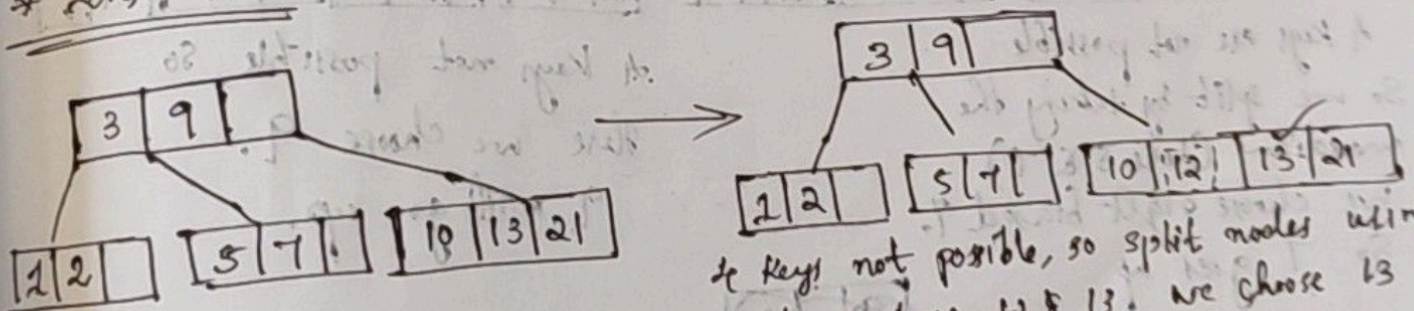
* Insert 7:



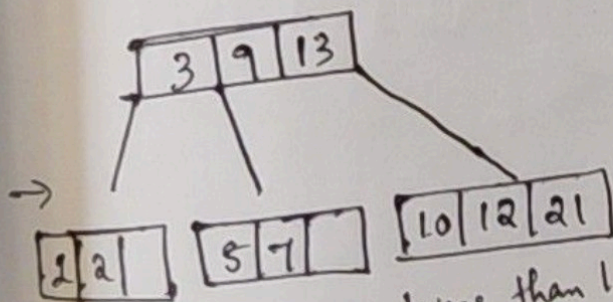
* Insert 10:



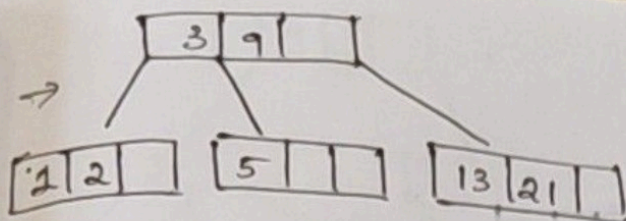
* Insert 12:



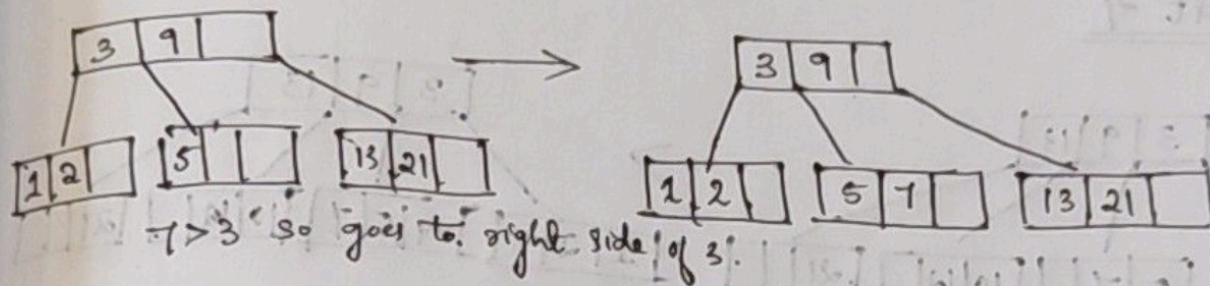
4 keys not possible, so split nodes using medians here 12 & 13, we choose 13 [right biased], then 13 goes up.



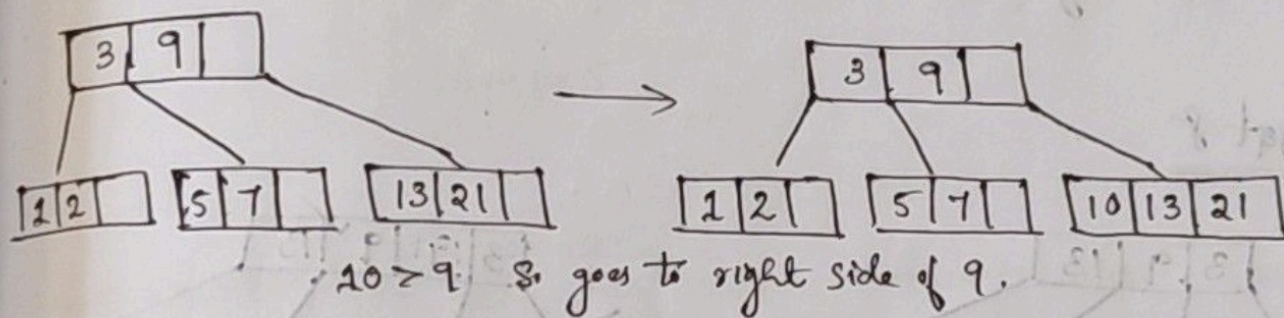
Here 10 & 12 are lesser than 13, so we create new children node & place 10 & 12 into that



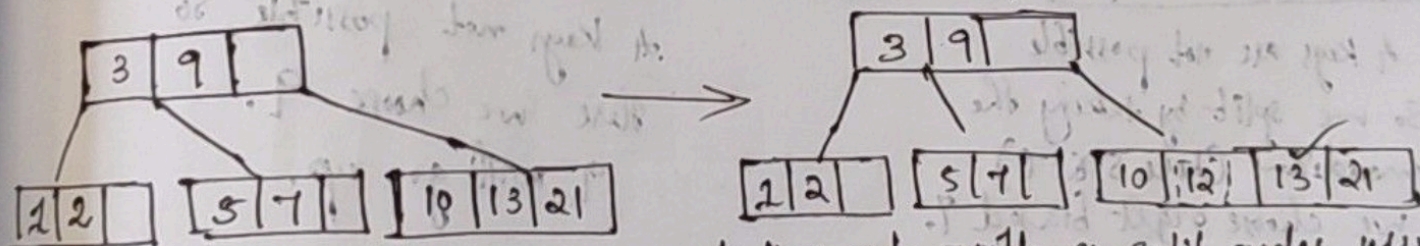
* Insert 7:



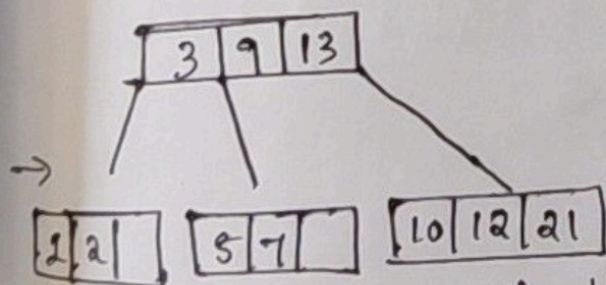
* Insert 10:



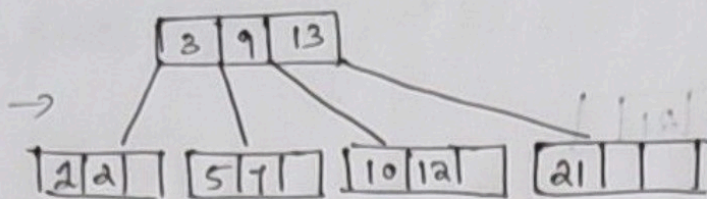
* Insert 12:



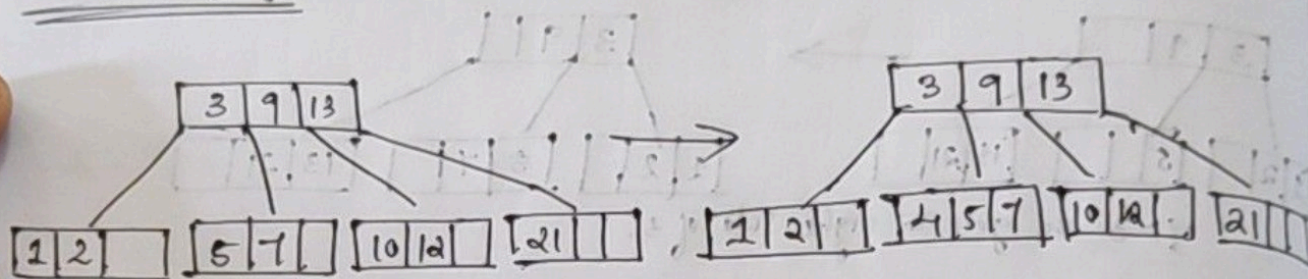
Keys not possible, so split nodes using medians here 12 & 13, we choose 13 [right biased], then 13 goes up.



Here 10 & 12 are lesser than 13, so we create new children node & place 10 & 12 into that

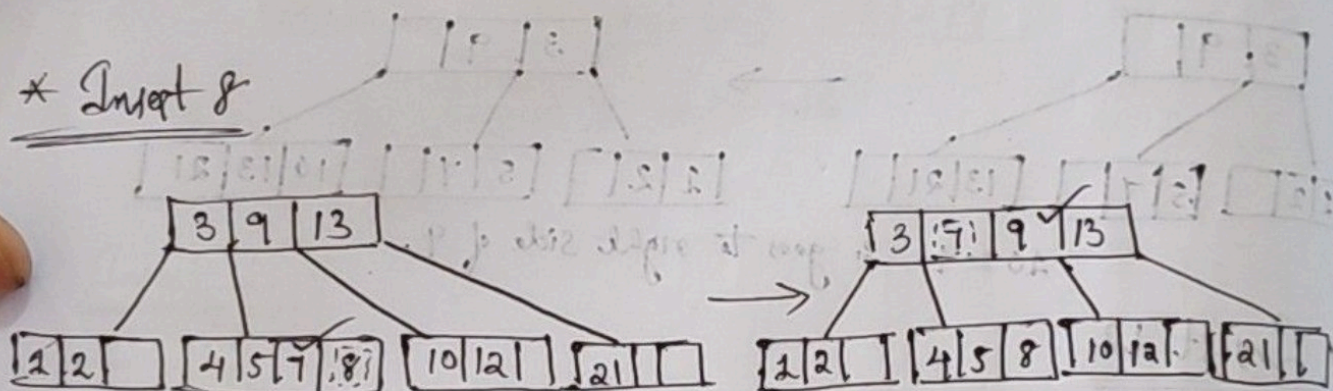


* Insert 4



4 > 3, so goes to right side node of 3.

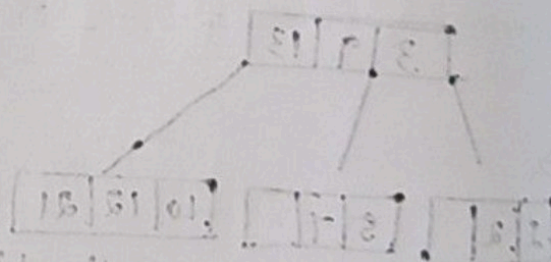
* Insert 8

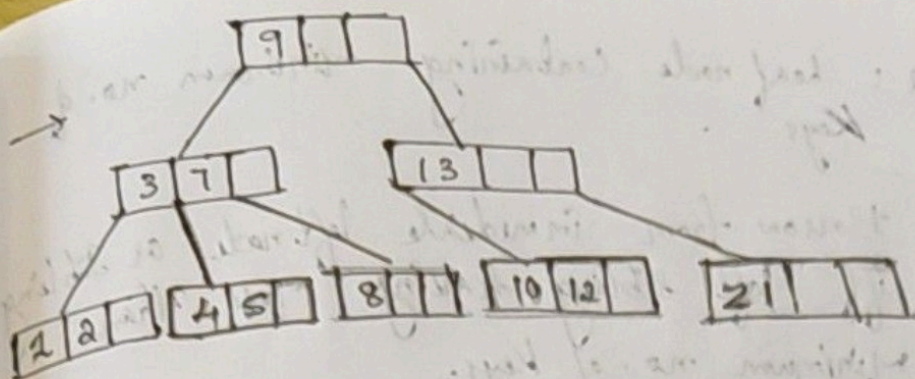


4 keys are not possible
 So we split by taking the
 median (here 5 & 7)
 we choose right biased 7.
 So 7 goes up



4 keys not possible, so
 here we choose 9.
 9 will go up.





B-Tree Deletion

Step 1: If target key is in leaf node

eg: $m=5$
 $\text{Keys} = 5 - 1 = 4$
 $\text{min children} = 5$

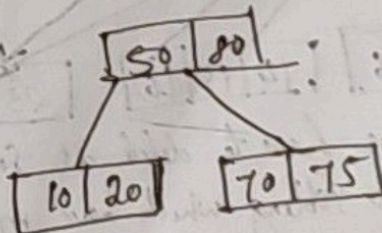
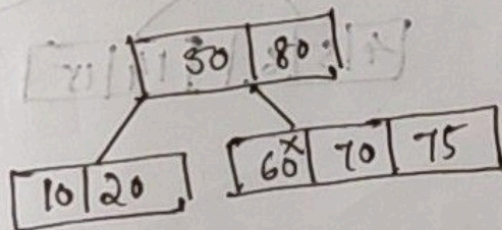
~~Step 1~~ Case 1: Leaf node containing more than minimum no. of keys, simply delete that key

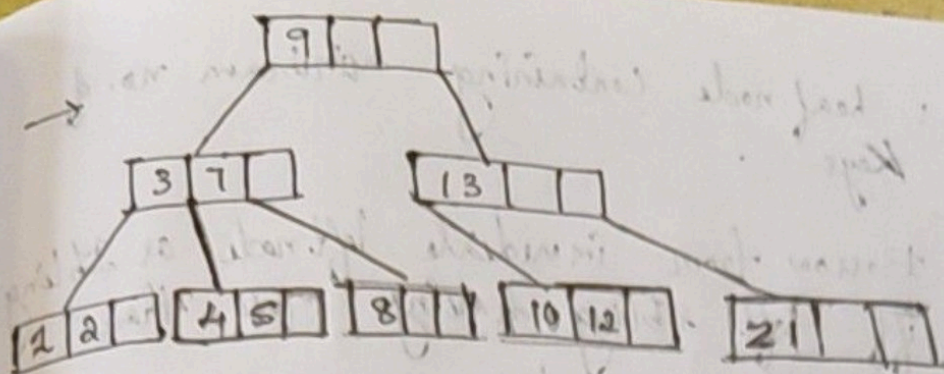
eg: $m=5$

$$\text{min keys} = \frac{5}{2} - 1$$

$$(2.5 - 1)$$

$$= 3 - 1 = 2 //$$





B-Tree Deletion

Step 1: If target key is in leaf node

eg: $m=5$

$$\text{Keys} = 5 - 1 = 4$$

$$\text{min children} = 5$$

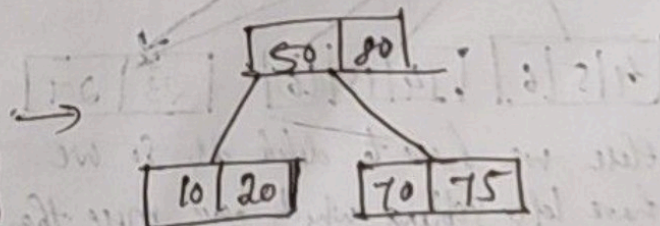
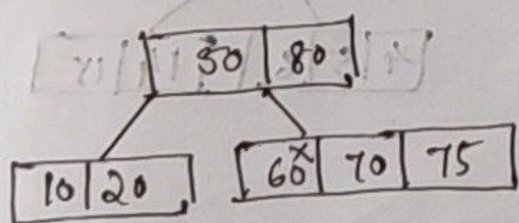
~~Step 2~~ Case 1: Leaf node containing more than minimum no. of keys, simply delete that key

eg: $m=5$

$$\text{min keys} = \frac{5}{2} - 1$$

$$(2.5 - 3)$$

$$= 3 - 1 = 2 //$$



→ Case 2: leaf node containing minimum no. of keys

i) → Borrow from immediate left node / or sibling iff left sibling having more than minimum no. of keys.

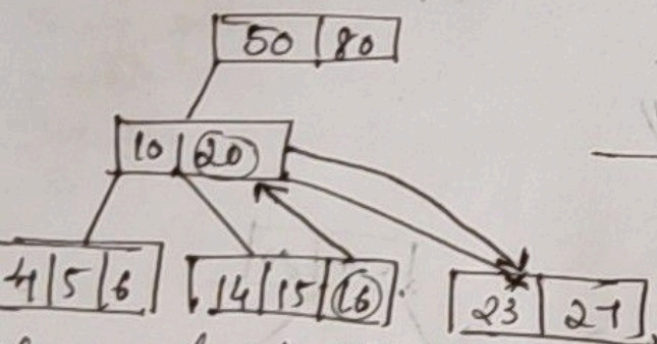
ii) → Borrow from immediate right node or sibling of right node having more than minimum no. of keys.

iii) ~~Case 3~~ → Neither the left sibling nor the right sibling having more than minimum no. of keys, therefore, perform merge operation.

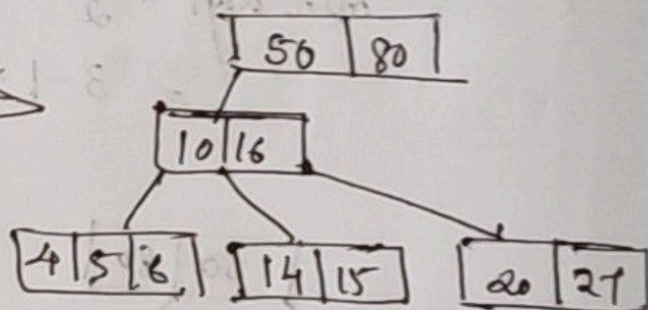
~~Case 3~~ ii) ^{Binary} Eg:

$$m=5$$

$$\min=2, \max=m-1=4$$



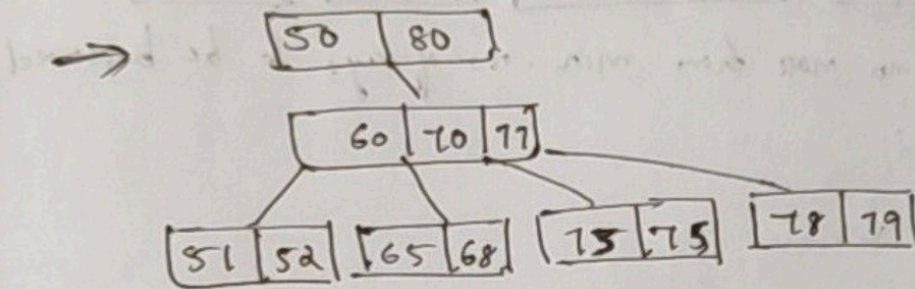
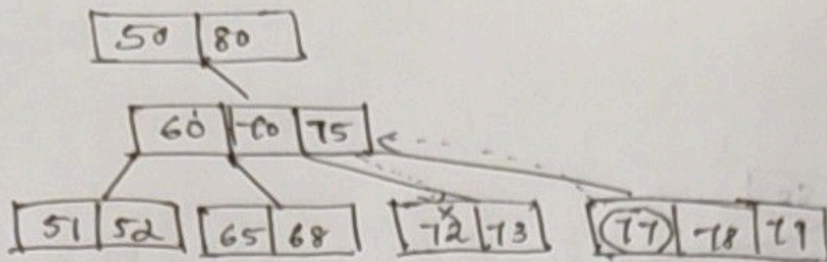
here we have to delete 23, so we have left sibling which have more than min no. of keys so we borrow it



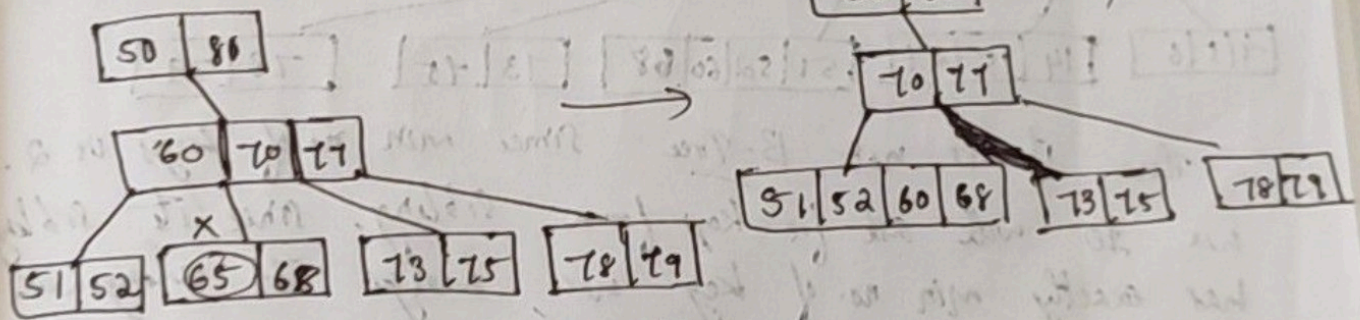
23 or 20, 26 or 27

Delete 72

eg:

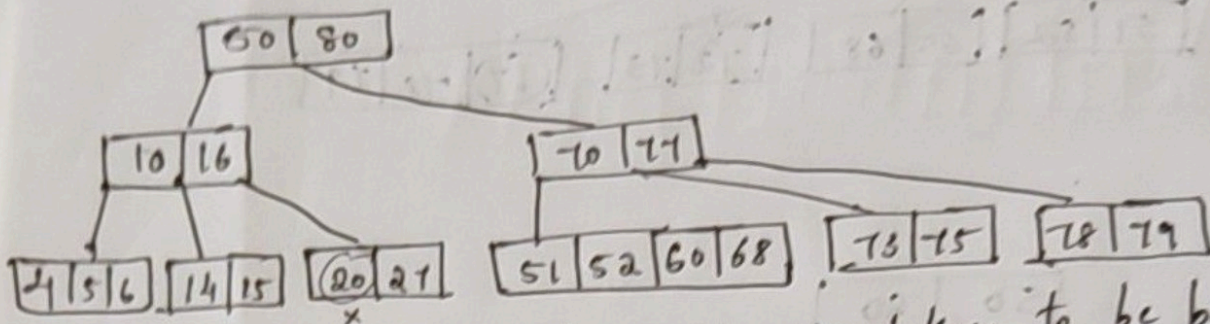


iii) Eg ⇒ merging

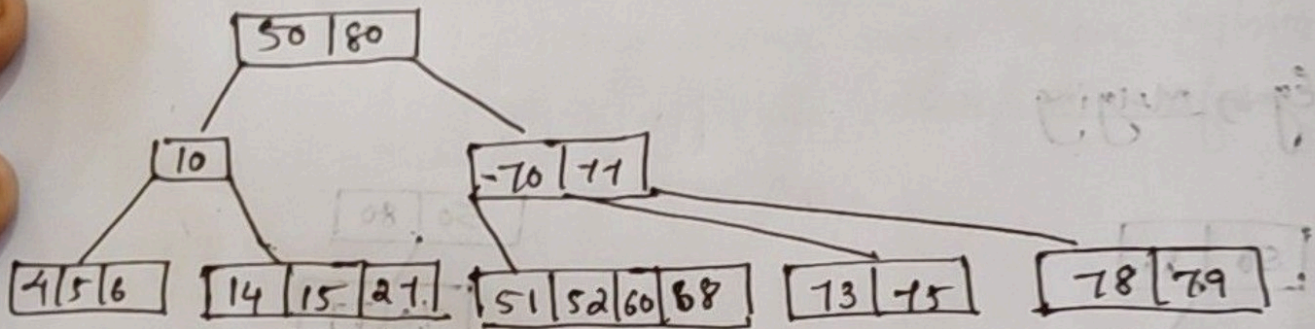


merge with left sibling or right sibling along with its parent

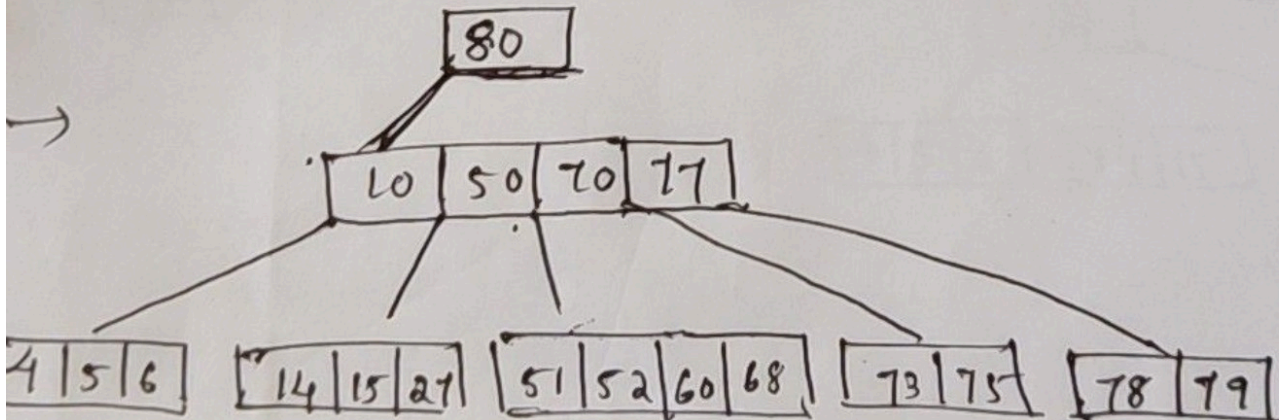
Eg



Here there is no more than min. no. of keys to be borrowed
so we merge.



Here it is not B-Tree since min no. of keys is 2.
we 10 will ask for keys from sibling. Since its sibling
has exactly min no. of key we merge it along with
its parent.



* Step 2: If the target key is internal node.

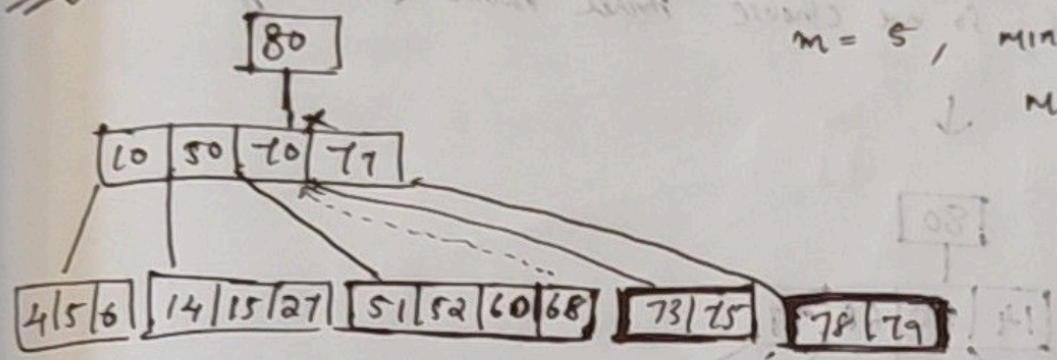
i) $\rightarrow$ Inorder predecessor: largest value from left side

ii) $\rightarrow$ Inorder Successor: smallest value from right side.

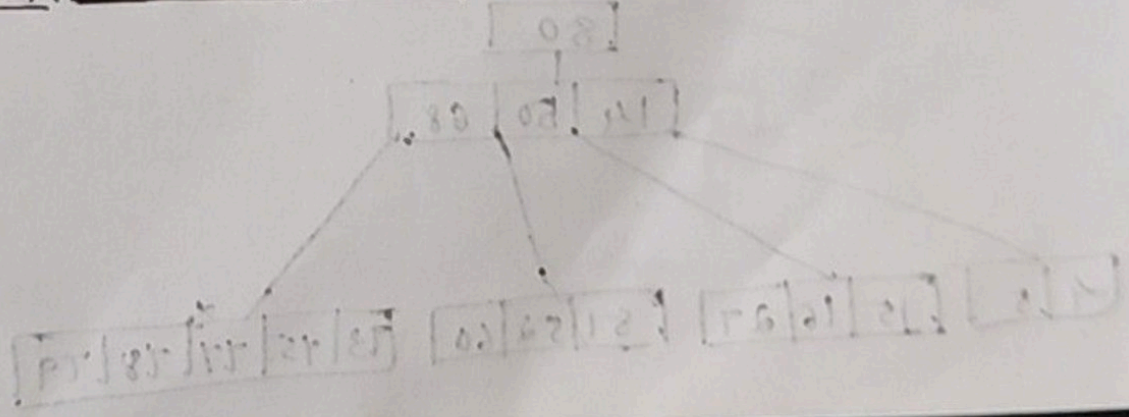
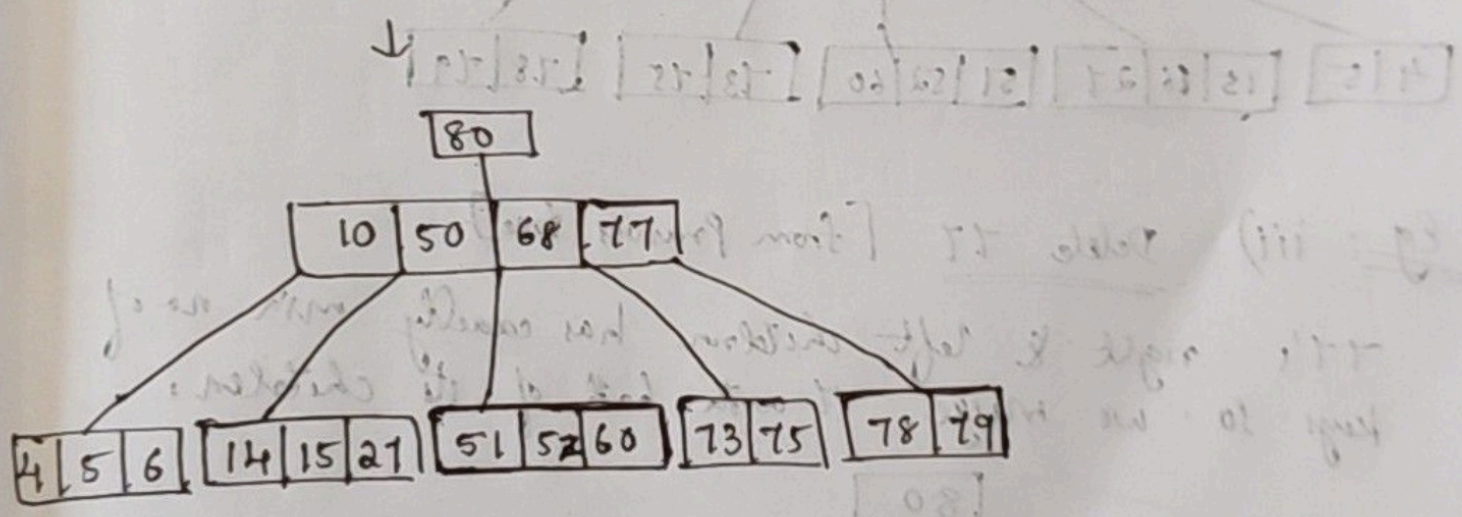
iii) $\rightarrow$ Neither the left node nor the right node have more than min no. of keys.

Eg: i) Delete 60

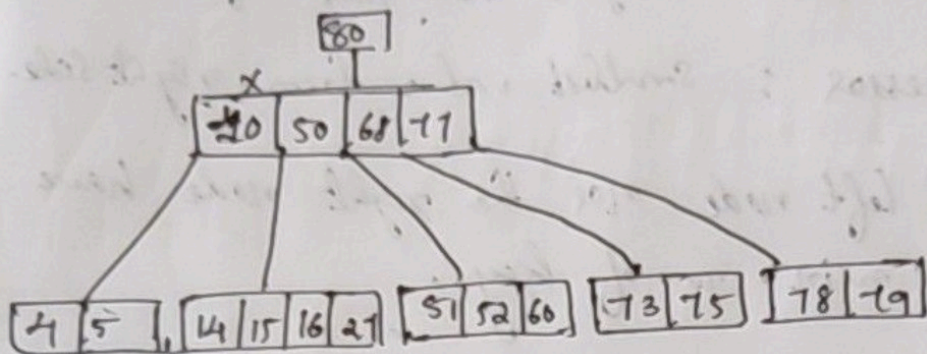
$m = 5$, min no. of keys = $\frac{m}{2} - 1 = 2$
 max no. of keys = $m - 1 = 4$



Replace with Inorder predecessor.

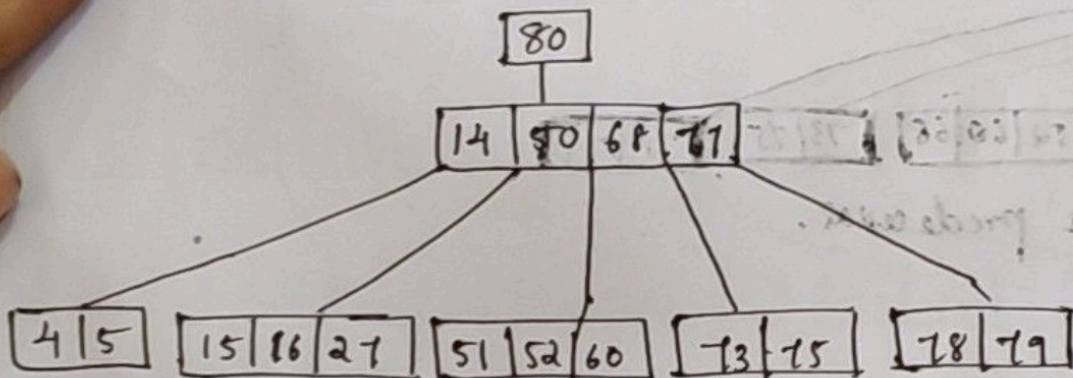


Eg.ii) Delete 10



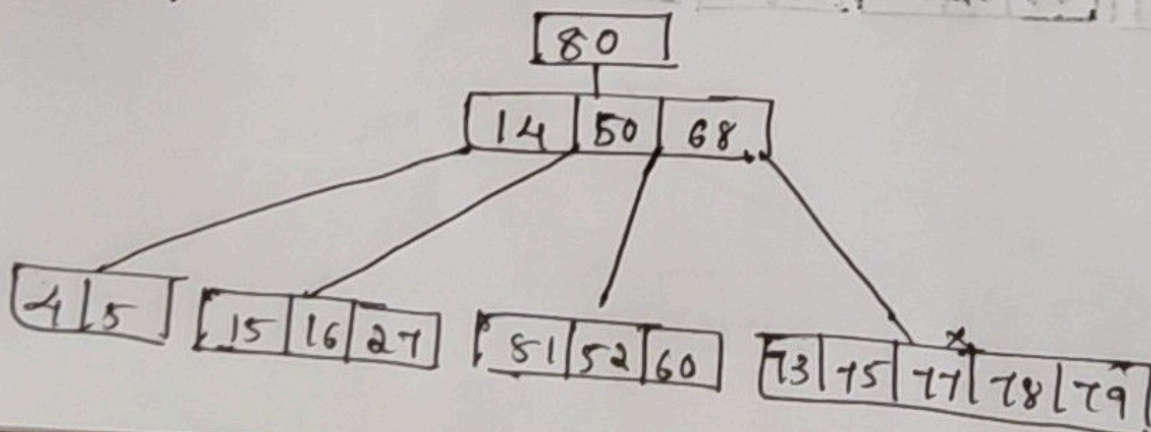
Here replace 10 with inorder successor. Since right side has max no. of keys, so we choose small value from right.

↓

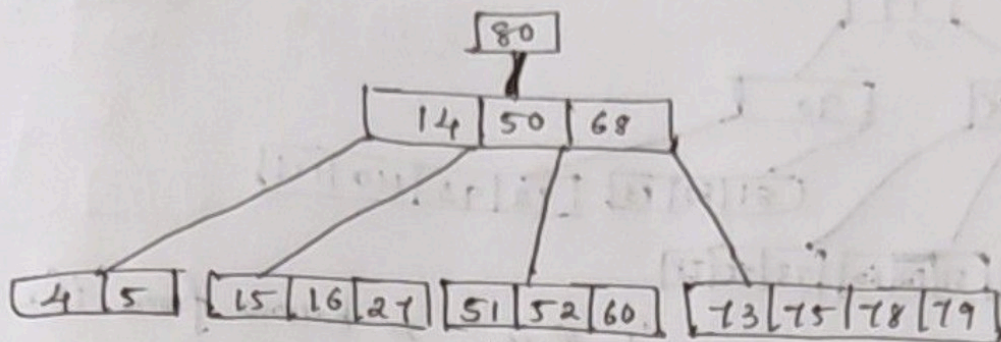


Eg: iii) Delete 77 [from previous tree]

77's right & left children has exactly min no. of keys so we merge 77 with both of its children.

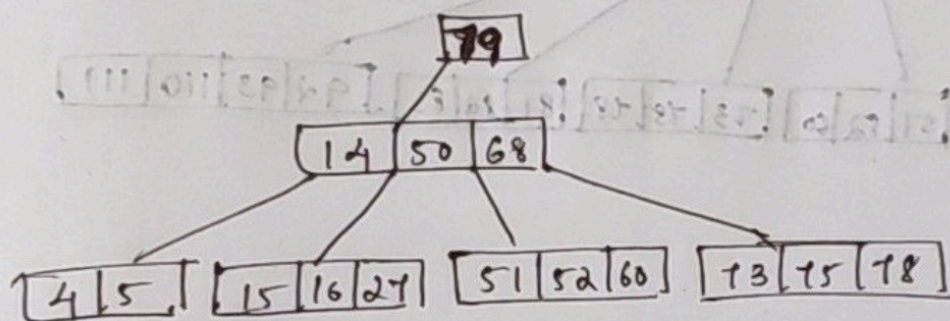


Then we can simply delete 77, since now it is in leaf node.

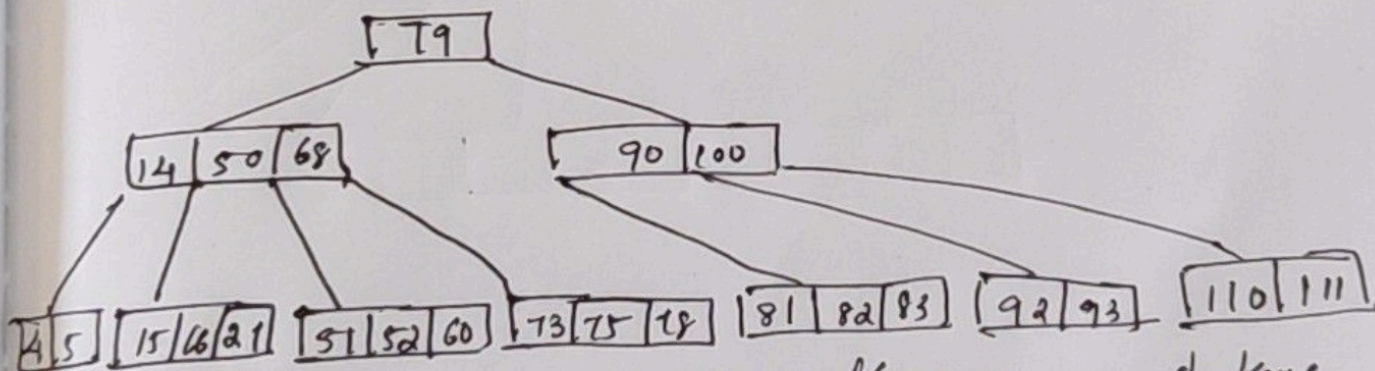


Eg: Delete 80 [root node]

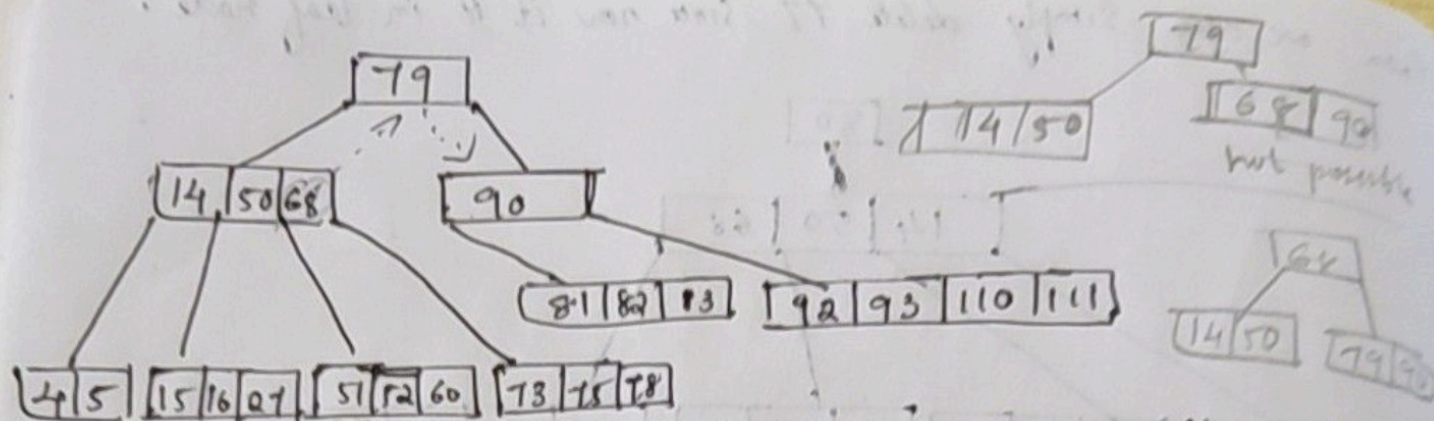
To delete root node replace with inorder predecessor or inorder successor. Here there is only left child for root node so we choose largest value from left side. Here it is 79.



Eg: Delete 100



Here 100 has children with exactly min no. of keys, so merge 100 with its children & then delete 100 simply.



Here min no. of key are not there. so borrow from the sibling.

